

```

# =====
# IRIS FLOWER CLASSIFICATION USING NAIVE BAYES
# Google Colab Ready
# =====

# Import libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# -----
# 1. Load Iris Dataset
# -----

iris = load_iris()

X = iris.data
y = iris.target

print("Feature Names:")
print(iris.feature_names)

print("\nTarget Names:")
print(iris.target_names)

print("\nDataset Shape:", X.shape)

# -----
# 2. Split Dataset
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data :", X_test.shape)

# -----
# 3. Create Naive Bayes Classifier
# -----

model = GaussianNB()

# -----
# 4. Train the Model
# -----

model.fit(X_train, y_train)

# -----
# 5. Predict Test Data
# -----

y_pred = model.predict(X_test)

# -----
# 6. Display Predictions
# -----

```

```

print("\n=====")
print("IRIS FLOWER CLASSIFICATION RESULTS")
print("=====")

print("\nActual\t\tPredicted")

for actual, predicted in zip(y_test, y_pred):
    print(
        f"{iris.target_names[actual]:12s}\t"
        f"{iris.target_names[predicted]}"
    )

# -----
# 7. Calculate Accuracy
# -----

accuracy = accuracy_score(y_test, y_pred)

print("\n=====")
print("ACCURACY")
print("=====")

print("Accuracy =", accuracy)
print("Accuracy (%) =", accuracy * 100)

# -----
# 8. Confusion Matrix
# -----

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# -----
# 9. Display Confusion Matrix as Heatmap
# -----

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.title("Iris Flower Classification - Naive Bayes")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()

# -----
# 10. Classification Report
# -----

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# -----
# 11. Predict a New Flower
# -----

# Measurements:
# [sepal length, sepal width, petal length, petal width]

```

```
new_flower = np.array([[5.1, 3.5, 1.4, 0.2]])

prediction = model.predict(new_flower)

print("\n=====")
print("NEW FLOWER PREDICTION")
print("=====")

print("Input measurements:", new_flower[0])
print("Predicted flower:", iris.target_names[prediction[0]])
```

```

Feature Names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Target Names:
['setosa' 'versicolor' 'virginica']

Dataset Shape: (150, 4)

Training Data: (120, 4)
Testing Data : (30, 4)

=====
IRIS FLOWER CLASSIFICATION RESULTS
=====

Actual      Predicted
setosa      setosa
virginica   virginica
versicolor  versicolor
versicolor  versicolor
setosa      setosa
versicolor  versicolor
setosa      setosa
setosa      setosa
virginica   virginica
versicolor  versicolor
virginica   virginica
virginica   virginica
virginica   virginica
versicolor  versicolor
setosa      setosa
setosa      setosa
setosa      setosa
versicolor  versicolor
versicolor  versicolor
virginica   virginica
setosa      setosa
virginica   virginica
versicolor  versicolor
versicolor  versicolor
virginica   virginica
versicolor  versicolor
versicolor  versicolor
setosa      setosa
virginica   virginica
setosa      setosa

=====
ACCURACY
=====
Accuracy = 0.9666666666666667
Accuracy (%) = 96.66666666666667

Confusion Matrix:
[[10 0 0]

```